BSc (Hons) in Computer Science
SE3112: Advanced Software Engineering
Year 3 · Semester 1 · 2026

Source Code : Lost and Found App ASE
CI/CD Pipeline : testing-pipeline-folder
Github actions runs : ase-testing-pipeline.yml

Back2U Lost & Found App - Comprehensive Microservice Testing Pipeline (SE3112) (Group 19)

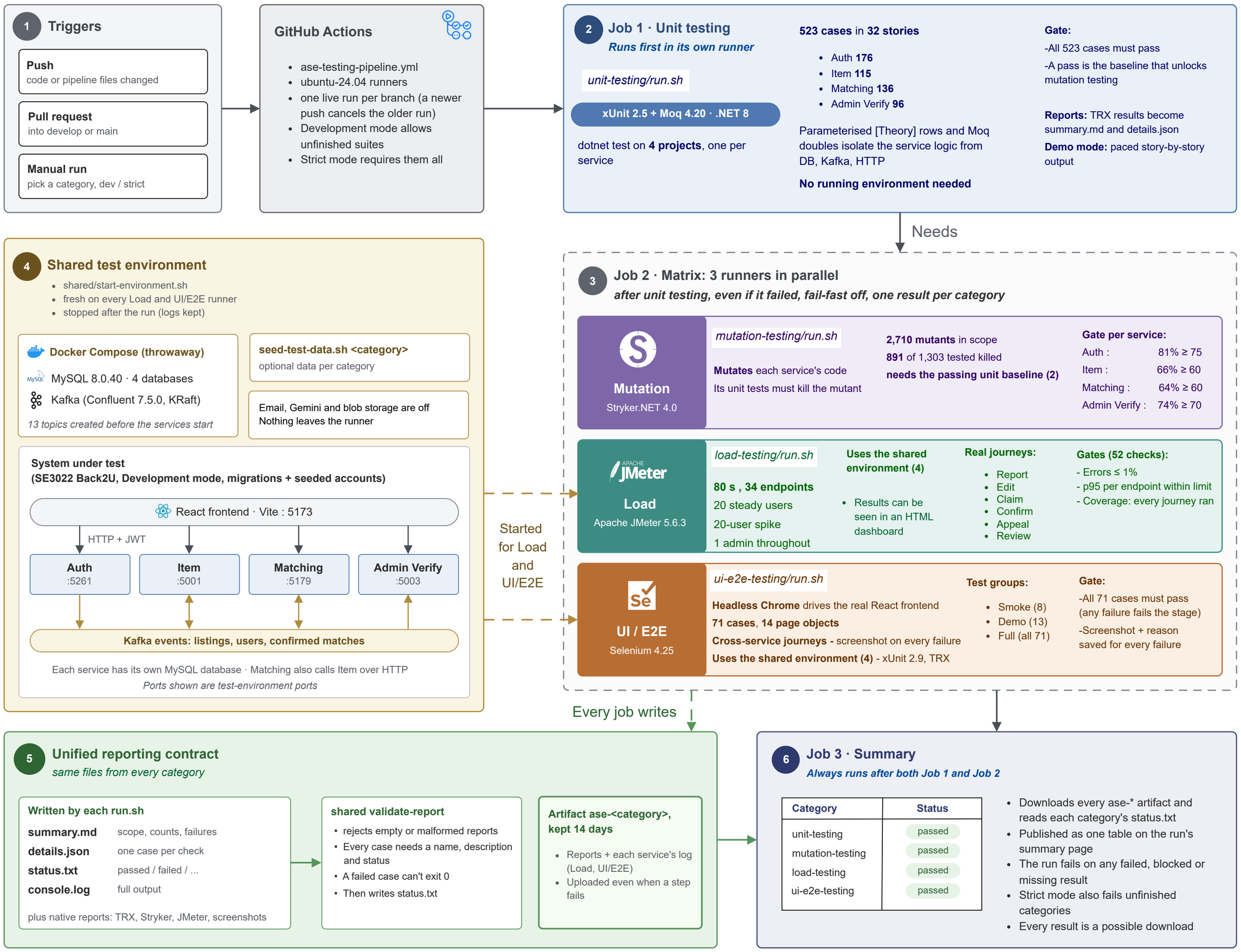
One GitHub Actions workflow runs four testing paradigms against the SE3022 microservices (Auth, Item, Matching, Admin Verify): each in its own runner, with a fresh environment where needed, and one shared reporting contract. Numbers (1) to (6) match the explanations below the diagram.

IT24103447 - Ranasinghe A. B.
Unit testing & mocking · CI workflow

IT24103457 - Karunaratne J.M.N.V.B
Mutation testing

IT24610328 - Wattegama B. C.
Load & performance testing

IT24103581 - Lalanga J. A. L.
UI / E2E testing



How the pipeline works

Every push or pull request that touches the services, the frontend or the pipeline starts one workflow. It checks the code at four levels, from the inside out: **unit tests** check each service's logic in isolation, **mutation testing** checks that those unit tests really catch faults, **load testing** checks the running system under many concurrent users, and **UI/E2E testing** checks complete user journeys through the real frontend. Each level is owned by one member, runs on its own machine, and reports in the same format, so one table shows the result of the whole run.

1 Triggers and workflow

- **Push** (any branch) or **pull request** into develop or main, when files under services/, frontend/, testing-pipeline/ or the workflow change.
- **Manual run**: pick one category or all, and a mode. **Development** lets unfinished suites report "not implemented". **Strict** requires every selected suite to run and pass. Real failures fail both.
- Runs on fresh ubuntu-24.04 GitHub runners. Only one run per branch is live. A newer push cancels the older one.

2 Job 1: unit testing and mocking

- **xUnit 2.5 + Moq 4.20** on .NET 8 and one test project per service. **523 cases in 32 stories** (Auth 176, Item 115, Matching 136, Admin Verify 96).
- Parameterised [Theory] cases cover boundaries and input rules, Moq test doubles replace repositories, Kafka publishers and HTTP clients, so only the service logic is tested.
- Runs first and needs no environment. Its result is the **baseline,** Mutation testing runs only after a passing baseline. A demo mode paces the output story by story.

3 Job 2: three stages in parallel

After unit testing finishes (pass or fail), a matrix starts three runners at once, with fail-fast off, so one stage failing never hides another's result.

Mutation (Stryker.NET 4.0). Makes small changes ("mutants") to each service's code and runs that service's unit tests against each one; a test must fail to "kill" it. One run per service, each gated on its own score, set just under its measured baseline: Auth 81% (gate 75), Item 66% (60), Matching 64% (60), Admin Verify 74% (70). 2,710 mutants in scope. Untested code is reported as a coverage gap, not hidden. No environment needed.

Load (Apache JMeter 5.6.3). 80 s against the running system. 20 steady users with think time, a 20-user spike at 60 s with none, and one admin throughout. Users report, edit, photograph, resolve and delete listings, claim and confirm matches with partner accounts and send appeals, while the admin reviews spam and decides appeals: 34 endpoints over all four services. Judged on error rate (≤ 1%), p95 per endpoint and coverage (every journey ran): 52 checks.

UI/E2E (Selenium WebDriver 4.25 + xUnit 2.9). Headless Chrome drives the real React frontend, and through it all four services. 71 cases built on 14 page objects, grouped Smoke (8), Demo (13) and Full. Each test arranges its own data through the API and saves a screenshot when it fails.

4 Shared test environment

- shared/start-environment.sh starts throwaway **MySQL 8.0.40** and **Kafka (Confluent 7.5.0)** in Docker Compose, creates the four databases and the 13 Kafka topics, then builds and starts Auth, Item, Matching and Admin Verify in Development mode (migrations run, test accounts seeded) and the React frontend.
- Started fresh on every Load and UI/E2E runner, never shared or reused, and stopped afterwards with the logs kept. A category can add its own data through seed-test-data.sh.
- Email, Gemini image processing and blob storage are switched off, so no test reaches a real service or person.

5 Unified reporting contract

- Each category has exactly one entry point, its run.sh, which CI and local runs both call. It writes summary.md (scope, counts, failures, measurements) and details.json (one case per check, with a description); the runner adds status.txt and console.log.
- A shared validate-report rejects missing or malformed reports and a failed case reported with exit code 0.
- Native reports travel alongside: TRX files, Stryker HTML, the JMeter dashboard, UI screenshots. Each category uploads an ase-<category> artifact (with service logs for Load and UI/E2E), kept 14 days.

6 Job 3: summary and final result

Always runs last. It collects every artifact, publishes one table (category and status) on the run's summary page, and sets the final result: any failed, blocked or missing category fails the run, and in strict mode so does an unfinished one.

Running it locally

The same scripts run on a developer machine (WSL2 or Linux with Docker, .NET 8, Node 22; Java 17 for load):

```
bash testing-pipeline/shared/start-environment.sh start
bash testing-pipeline/<category>/run.sh
bash testing-pipeline/shared/start-environment.sh stop
```

Unit and mutation testing need no environment. Each category folder has a README with its own commands, demo options and results.

Tools and versions

Area	Tools
CI	GitHub Actions, ubuntu-24.04, Bash, Node 22.14 (reporting)
Unit	.NET 8, xUnit 2.5, Moq 4.20
Mutation	Stryker.NET 4.0 (dotnet tool, pinned)
Load	Apache JMeter 5.6.3 (checksum-verified, cached), Java 17
UI/E2E	Selenium WebDriver 4.25, xUnit 2.9, Chrome
Environment	Docker Compose, MySQL 8.0.40, Kafka (Confluent 7.5.0)

Design principles

- **Isolation:** every stage on its own runner and, where needed, its own fresh environment, so results are repeatable and independent.
- **One entry point per stage:** the same run.sh locally and in CI, so a local pass means the same checks.
- **Fail loudly:** thresholds, mutation gates, coverage checks and report validation turn quality problems into a failed run, not a footnote.
- **No side effects:** nothing touches the deployed system, real users or real email.